Rapport de Projet : API REST de Gestion d'Étudiants

1. Description du projet et Cas d'Utilisation

Ce projet consiste à développer le back-end d'une application permettant la gestion complète d'une liste d'étudiants. L'interface applicative (API REST) développée en Python à l'aide du framework **Flask** interagit avec une base de données MySQL.

Les cas d'utilisation principaux couvrent l'intégralité du cycle CRUD (Create, Read, Update, Delete) par un administrateur système ou une application cliente :

- **Consulter les étudiants :** Récupérer la liste complète des étudiants inscrits dans la base.
- **Consulter un étudiant :** Afficher les informations détaillées d'un étudiant spécifique via son identifiant (ID).
- **Ajouter un étudiant :** Insérer un nouveau profil (nom, prénom, email, téléphone) dans la base de données.
- **Modifier un étudiant :** Mettre à jour les coordonnées d'un étudiant existant.
- **Supprimer un étudiant :** Effacer l'enregistrement d'un étudiant ciblé.

2. Description de l'API (Format Swagger)

Voici la spécification détaillée des routes HTTP exposées par l'API (basée sur la dernière version opérationnelle) :

Méthode	Endpoint (Route)	Description	Codes HTTP (Retours)
GET	`/v4/etudiants/`	Retourne la liste de tous les étudiants au format JSON.	**200**: Succès **401**: Non autorisé **500**: Erreur serveur
GET	`/v4/etudiants/{id}`	Retourne les détails de l'étudiant correspondant à l'identifiant fourni.	**200**: Succès **404**: ID invalide/Introuvable **401**: Non autorisé
POST	`/v4/etudiants/`	Ajoute un nouvel étudiant. Nécessite un corps de requête JSON (nom, prénom, email, téléphone).	**201**: Ajout OK **400**: Requête invalide **401**: Non autorisé

Méthode	Endpoint (Route)	Description	Codes HTTP (Retours)
PUT	`/v4/etudiants/{id}`	Met à jour les informations d'un étudiant existant (identifié par son ID).	**200**: Modification OK **404**: ID invalide **401**: Non autorisé
DELETE	`/v4/etudiants/{id}`	Supprime définitivement un étudiant de la base de données.	**200**: Suppression OK **404**: ID invalide **401**: Non autorisé

3. Récapitulatif des évolutions (Versions de l'API)

- **Version 1 (api_v1.py) :** Implémentation fonctionnelle de base du CRUD. Configuration des routes Flask, exécution des requêtes SQL directes via le connecteur MySQL, utilisation du versioning dans l'URL et renvoi des codes de statut HTTP standard.
- **Version 2 (api_v2.py) :** Ajout de la gestion structurée des erreurs (blocs `try/except`) pour éviter les crashs de l'application (ex: ID inexistant). Optimisation des requêtes via l'usage intensif des f-strings en Python. Amélioration du contrôle des privilèges d'accès à la base locale.
- **Version 3 (api_v3.py) :** Intégration d'un mécanisme d'authentification (Basic Auth). Création d'une fonction `login()` vérifiant les credentials HTTP dans une table `user` de la base de données pour sécuriser l'accès aux routes.

4. Diagramme de classes (Classe Database)

Dans la version 4, le code a été refactorisé selon le paradigme orienté objet. Le routage a été séparé de l'accès aux données. Voici la représentation de la classe `Database` (générée par pyreverse) :



5. Fiche de tests Postman

Tableau récapitulatif des tests d'intégration menés sur la v4 de l'API (avec Authentification) :

Cas de Test	Méthode	Endpoint ciblé	Code Attendu	Résultat Obtenu
Récupérer tous les étudiants (sans Auth)	GET	`/v4/etudiants/`	401 Unauthorized	**Succès** (401)
Récupérer tous les étudiants (Auth valide)	GET	`/v4/etudiants/`	200 OK	**Succès** (200, liste JSON)
Créer un étudiant (Auth valide, payload correct)	POST	`/v4/etudiants/`	201 Created	**Succès** (201)
Chercher un étudiant avec ID inexistant	GET	`/v4/etudiants/9999`	404 Not Found	**Succès** (404, "id invalide")
Supprimer un étudiant existant	DELETE	`/v4/etudiants/5`	200 OK	**Succès** (200)

6. Conclusion et Améliorations Cybersécurité

L'API développée répond aux exigences fonctionnelles du cahier des charges (CRUD, interfaçage base de données, modularité POO). Néanmoins, dans une optique de mise en production, l'application présente d'importantes vulnérabilités de cybersécurité qui doivent être corrigées :

- **Injections SQL :** L'utilisation actuelle des f-strings Python pour concaténer directement les variables utilisateur dans les requêtes SQL (ex: `f"INSERT INTO... VALUES ('{data['nom']}')"`) expose la base de données aux injections SQL. *Solution : Remplacer les f-strings par des requêtes préparées (requêtes paramétrées avec `%s`).*
- **Stockage des mots de passe :** Les mots de passe utilisateurs sont actuellement stockés et comparés en clair dans la table `user`. *Solution : Stocker des empreintes cryptographiques (hash) en utilisant une bibliothèque comme `bcrypt` ou `argon2`.*
- **Authentification et Transport :** L'authentification Basic transmet les identifiants codés en Base64 à chaque requête, ce qui est très vulnérable si le trafic est intercepté. *Solution : Implémenter le protocole HTTPS (certificat TLS) pour chiffrer le tunnel, et privilégier une authentification par jeton (ex: JWT - JSON Web Tokens) plutôt que Basic Auth.*
- **Failles XSS (Cross-Site Scripting) :** Les données JSON reçues lors du POST et PUT ne sont ni nettoyées ni validées. Si une application front-end affiche ces données sans échappement, des scripts malveillants pourraient être exécutés. *Solution : Assainir (sanitize) les entrées au niveau du back-end.*